Git

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

Brief overview / introduction

- **Local vs remote repos**
  - A **local repository** is a repository that is stored on a computer.
  - A **remote repository** is a repository that is hosted on a hosting service.
- **Local repositories**: A local repository is represented by a hidden directory called .git that exists within a project directory. It contains all the data on the changes that have been made to the files in a project.

  You should never touch any files or directories inside your .git directory. Doing this could have undesirable consequences for your repository. You should never delete this directory unless you want to delete your repository.
- **git init**: git init creates a new empty Git repository, or reinitializes an existing one. In the normal case, it creates a .git/ directory containing Git's object database, refs, tags, config, hooks/templates, and other repository metadata. Running git init again inside an existing repository is safe; Git will not overwrite existing repository data.

```
git init [-q | --quiet]
[--bare]
[--template=<template-directory>]
[--separate-git-dir <git-dir>]
[--object-format=<format>]
[--ref-format=<format>]
[-b <branch-name> | --initial-branch=<branch-name>]
[--shared[=<permissions>]]
[<directory>]
```

  - -q Suppress normal output. Git will only print errors and warnings. This is useful in scripts where you do not want the usual "Initialized empty Git repository..." message.
  - –bare Creates a bare repository. A bare repository has no working tree; it is essentially just the repository database itself. Instead of storing Git metadata inside .git/, the repository directory itself contains things like objects/, refs/, HEAD, and config. Git's repository-layout documentation describes a bare repository as one "without its own working tree," typically used for exchanging histories by pushing and fetching.
  - –template Tells Git where to copy template files from when initializing the repository. Templates can include default hook samples, info files, exclude patterns, and initial repository structure. Git chooses the template directory in this order: the –template argument, then $GIT_TEMPLATE_DIR, then init.templateDir, then the default template directory.

    This is useful if you want every new repo to start with the same hooks or config-related files.
  - –separate-git-dir Stores the actual Git repository data somewhere other than .git/ in the working directory. Instead of creating a real .git/ directory, Git creates a .git text file that points to the actual repository directory. If used while reinitializing, Git moves the repository to the specified path

Stores the actual Git repository data somewhere other than .git/ in the working directory. Instead of creating a real .git/ directory, Git creates a .git text file that points to the actual repository directory. If used while reinitializing, Git moves the repository to the specified path

- –object-format Chooses the hash algorithm used for Git objects. Valid values are currently sha1 and, if supported by your Git build, sha256. sha1 is the default. Git's documentation also notes that SHA-256 repositories and SHA-1 repositories currently do not interoperate directly.
- –ref-format Chooses how Git stores refs, such as branches and tags. Valid values are files and reftable. The default is files, meaning loose ref files plus packed-refs. reftable uses the newer reftable storage format.
- -b Sets the name of the initial branch in the new repository. These two forms are equivalent. For example, this creates the repo with main as the initial branch name:
- –shared Initializes the repository with permissions suitable for sharing among multiple users. This sets core.sharedRepository, causing files under $GIT_DIR to be created with the requested permissions. If –shared is not specified, Git uses the normal permissions determined by your system umask.
- Tells Git where to initialize the repository. If the directory does not exist, Git creates it. If it does exist, Git initializes the repository inside it.

- **The areas of Git**: There are four important areas to be aware of when you are working with Git:

  1. **Working directory**: The actual files and folders you see and edit in your project directory.
  2. **Staging area**: The intermediate area where you choose which changes will go into the next commit.

     The staging area is also called the index. When you run

     ```
     1  git add main.c
     ```

     you copy the current version of main.c from the working directory into the staging area.

     This lets you prepare a commit carefully. For example, you might edit three files but stage only one of them.

     You can inspect staged changes with:

     ```
     0  git diff --staged
     ```

  3. **Commit history**: The sequence of commits that record the project's past states. A commit is a snapshot of the staged files, plus metadata such as author, date, commit message, and a pointer to the previous commit.

     A — B — C

     Here, A, B, and C are commits. The latest commit on your branch is usually what HEAD points to.

You can view the commit history with

```
git log
```

And

```
git log --graph
```

4. **Local repository**: The Git repository stored on your own machine. In a normal project, the local repository is mainly the .git directory

   The .git/ directory contains Git's internal database: commits, branches, tags, object files, configuration, refs, and the staging area.

- **Working tree**: The working tree is the set of actual project files that git has checked out for you to edit. For

```
myproject/
main.c
README.md
src/
.git/
```

The working tree is

```
main.c
README.md
src/
```

- **git diff**: Git can show the difference between the working tree and the staging area with

```
git diff
```

shows changes in your working tree that are not staged. Answers:

"*What have I changed but not staged yet?*"

```
git diff --staged
```

Shows staged changes, meaning what would go into the next commit.

```
git diff HEAD
```

Shows all changes compared to the last commit, both staged and unstaged.

```
git diff commit1 commit2
```

Shows what changed between two commits

```
git diff main feature
```

Shows the difference between the snapshots pointed to by main and feature.

- **Tracked files**: A tracked file is a file Git already knows about.

  More precisely, a file is tracked if it is already in Git's repository history or currently in the index.

  A tracked file is under Git's management. It can be in states such as:

  1. unmodified
  2. modified
  3. staged
  4. deleted

- **git add -A**: Stages all additions, modifications, and deletions in the whole repository.

  ```
  git add -A
  ```

  stages all changes in the entire repository, regardless of what directory you are currently in.

  ```
  git add .
  ```

  stages changes only in the current directory and below.

- **git add –patch**: Lets you stage changes **piece by piece** instead of staging an entire file.

  ```
  git add --patch
  ```

  or

  ```
  git add -p
  ```

shows you each change hunk and asks whether you want to stage it. A hunk is a small contiguous block of changed lines. For example, if you edited two unrelated parts of the same file, Git may show them as two separate hunks.

- **git restore, revert, and reset**:
  - **Restore**: git restore is mainly for undoing changes in your working tree or staging area. Use it when you want to throw away uncommitted changes.

    ```
    git restore file.txt
    ```

    This discards unstaged changes to file.txt in your working tree and restores it to the version currently in the staging area.

    ```
    git restore --staged file.txt
    ```

    removes file.txt from the index, –staged can undo a git add

    ```
    git restore --worktree file.txt
    ```

    discards unstaged changes in the file and restores it from the index. Note that this is equivalent to

    ```
    git restore file.txt
    ```

    because –worktree is the default target for git restore.
  - **revert**: git revert undoes a commit by creating a new commit that applies the opposite changes.

    Use it when the bad commit is already part of shared history, such as a branch you pushed to GitHub.

    If commit **abc123** adds a line "*hello*", then

    ```
    git revert abc123
    ```

    creates a new commit that removes that line. The original commit still exists in history.
  - **Reset**: git reset moves the current branch pointer to another commit. Depending on the option, it may also change the staging area and working tree.

    Use it when you want to move your branch backward or rewrite local history.
- **Branches**: A branch in Git is a movable name that points to a commit. That is the core idea. Suppose history looks like this:

  A -- B -- C

  Suppose main is a branch, and points to commit C. When you make a new commit while on main, Git creates a new commit and moves main forward:

Branches let you work on a separate line of development without disturbing another line.

```
git switch -c feature-login
```

This creates a new branch called feature-login and switches to it. Now, both main and feature-login point to commit C. If you commit on feature-login, only that branch moves forward.

```
A -- B -- C
^
main
\
D -- E
^
feature-login
```

- **HEAD**: HEAD means "where you currently are." Usually, head points to a branch. If you make a commit, git moves the branch that HEAD is attached to

```
git branch feature-login
```

Creates a branch but does not switch to it.

```
git switch -c feature-login
git checkout -b feature-login
```

Both create the branch and switch to it

- **Merging branches**: Eventually, you usually want to bring the changes from one branch into another. Suppose you want to merge feature-login into main. First, switch to main.

```
git switch main
```

Then, merge

```
git merge feature-login.
```

```
A -- B -- C -------- M
\        /
D -- E
```

M is a merge commit. It combines the histories of main and feature-login. Sometimes Git can do a **fast-forward merge** instead:

```
A -- B -- C -- D -- E
^
main
```

Note that we can pass `--no-ff` to force a merge commit.

```
git merge --no-ff feature-login
```

After merging a branch, you can delete the local branch name:

```
git branch -d feature-login
```

This deletes the branch pointer, not necessarily the commits. If the commits are already reachable from another branch, such as main, they are still part of history.

```
git branch -D feature-login
```

This deletes the branch even if Git thinks it has unmerged work.

- **Squash merge vs regular merge**: A regular merge preserves the branch history. A squash merge takes the final changes from a branch but does not preserve that branch's individual commits in the target branch history.

  Suppose we have the history

```
A---B---C  main
\
D---E---F  feature
```

  A regular merge creates a new merge commit that has two parents: the current main commit and the tip of feature.

```
git switch main
git merge feature
```

```
A---B---C-------M  main
\         /
D---E---F  feature
```

The commits D, E, and F remain visible as separate commits in the project history. Git records that feature was merged into main.

Use a regular merge when you want to preserve the real development history, including all intermediate commits and the fact that a branch existed.

A squash merge takes the combined changes from D, E, and F, applies them to main, and then lets you commit them as one new commit.

```
git switch main
git merge --squash feature
git commit
```

```
A---B---C---S  main

D---E---F  feature
```

Here, *S* is a single new commit containing the same final changes as the whole feature branch.

But importantly, S is not a merge commit. It has only one parent: C.

Git does not record that feature was merged into main. The commits D, E, and F are not part of main's history.

After a squash merge, Git does not consider the feature branch "merged" in the same structural way.

```
git branch --merged
```

Will not show that the branch was merged.

- **Local vs remote branches**: A local branch exists in your repository. A **remote-tracking branch** represents the state of a branch on a remote, such as GitHub.

  origin/main means: My local record of what main looked like on the remote named origin.

  ```
  git fetch
  ```

  updates remote-tracking branches.

  ```
  git push origin feature-login
  ```

Git sends your local feature-login branch to the remote.

- **git fetch**: The git fetch command downloads the latest commits, files, and references from a remote repository into your local repository, but it does not merge or integrate them into your current working code.

  It essentially acts as a safe, "read-only" check that updates Git's internal tracking records to mirror what is happening on the server. Because it never alters your local branches or working files, it is completely safe to run at any time without risking merge conflicts.

- **Checkout vs switch**: Git checkout is the older, overloaded command. Switch is the newer, more specific command for switching branches.

```
git checkout main
git switch main
```

  Both switch you to the main branch. But checkout can do multiple unrelated things. It can also restore files (seen below).

- **Hunks**: In Git, a hunk is a contiguous block of changes inside a file. When Git shows a diff, it does not usually show the entire file. It shows only the changed regions, plus a little surrounding context. Each changed region is called a **hunk**.

```
diff --git a/file.txt b/file.txt
index 83db48f..f735c2d 100644
--- a/file.txt
+++ b/file.txt
@@ -1,5 +1,5 @@
line 1
line 2
-old line
+new line
line 4
line 5
```

  This part is the hunk:

```
@@ -1,5 +1,5 @@
line 1
line 2
-old line
+new line
line 4
line 5
```

  - The -old line means that line was removed.
  - The +new line means that line was added.

  The unchanged lines around it are context, included so Git and humans can understand where the change belongs.

```
@@ -1,5 +1,5 @@
```

is the **hunk header**. It means:

```
-1,5   old file: starting at line 1, show 5 lines
+1,5   new file: starting at line 1, show 5 lines
```

So the hunk describes a change from one version of the file to another.

- **Patches**: A patch is a text representation of changes. In Git, a patch usually means a **diff file** that describes how to change one version of files into another version.

  A patch is made out of one or more **hunks**.

  You can create a patch with

  ```
  git diff > changes.patch
  ```

  Someone else can apply it with

  ```
  git apply changes.patch
  ```

  That modifies their working tree using the changes described in the patch.

- **Merges**: A merge in git combines changes from one branch into another branch.

  ```
  git switch main
  git merge feature
  ```

  This means: Take the changes from feature and integrate them into main.

  A merge conflict happens when Git cannot automatically combine the changes. Suppose both branches changed the same line differently,

  ```
  // main.c
  int x = 10;
  ```

  ```
  // feature.c
  int x = 20;
  ```

  Git cannot possibly know which one you want, so it marks the conflict in the file.

```
<<<<<<< HEAD
int x = 10;
=======
int x = 20;
>>>>>>> feature
```

This means

```
HEAD side:    your current branch, main
other side:   the branch being merged, feature
```

You manually edit the file to the final version you want, then you stage the resolved file, and finish the merge with a commit.

- **Cherry pick**: git revert and git cherry-pick are almost opposites in spirit.

  git cherry-pick copies a commit's changes into your current branch. Use cherry-pick when you want to take one specific commit from another branch and apply it to your current branch.

- **Tags**: You can attach a name to a commit using a tag.

```
git tag v1.0 a3f4c91
```

  Now, v1.0 refers to that commit.

- **Stashing**: Git stash temporarily saves your uncommited changes somewhere inside git, then cleans your working tree. Use it when you are in the middle of work but need to switch branches, pull changes, or do something else without committing yet.

```
git stash
```

  This takes your current uncommitted changes and stores them in a stash entry.

  Then your working tree goes back to a clean state.

```
git stash list
```

  lists the stashes

```
git stash apply
```

  Brings back the most recent stash

```
git stash pop
```

Applies and removes from the list

```
git stash apply <stash name>
```

Applies a specific stash from the list.

```
git stash drop <name>
git stash clear
```

Removes stashes from the list, or clears the list.

```
git stash --staged
```

stashes only the staged changes. That means Git takes the changes currently in the index/staging area and puts them into a stash. Your unstaged working-tree changes are left alone.

```
git stash --keep-indexd
```

This stashes your unstaged changes, but keeps your staged changes in place.

In other words, the index is preserved.

- **Staging area manipulations**: Staging is represented by a directory **index** instead .git. The commands that
  - **git add**: Stages file contents. More precisely, it copies the current content of selected working-directory files into the index, preparing that content for the next commit. git add -p stages selected hunks interactively. git add -A stages additions, modifications, and deletions.
  - **git rm**: Removes paths from the index. By default, it removes them from both the working tree and the index. With –cached, it removes them from the index only, leaving the file in your working directory

    ```
    git rm file.c
    git rm --cached file.c
    ```

    Use git rm –cached when you accidentally started tracking a file but want to keep the local copy.
  - **git mv**: Moves or renames a file and updates the index to record that change. The change is staged, but it still needs to be committed.

```
git mv oldname.c newname.c
```

This is roughly equivalent to

```
mv oldname.c newname.c
git rm oldname.c
git add newname.c
```

- **git restore –staged**: Changes the index without necessarily changing the working directory. This is the modern command for "unstaging" something. By default, git restore –staged <path> restores the index version from HEAD.

```
git restore --staged file.c
```

You can also restore both the index and working tree:

```
git restore --staged --worktree file.c
```

- **git reset**: git reset can move HEAD, reset the index, and sometimes reset the working tree depending on the mode.

```
git reset file.c
```

Unstages file.c

```
git reset
```

Resets the whole index to HEAD, leaving the working directory alone

```
git reset --mixed HEAD~1
```

Moves HEAD and resets the index, but keeps working-directory changes. –mixed is the default reset mode. That is,

1. Move the current branch/HEAD back to HEAD∼1.
2. Reset the index to match the tree of HEAD∼1.
3. Leave the working tree alone.

```
git reset --hard HEAD
```

Resets both the index and working tree to HEAD

```
git reset --soft HEAD~1
```

moves HEAD back one commit, leaves index unchanged, leaves working tree unchanged.

- **git checkout**: git checkout is older and overloaded. It can affect the index in some forms but not others.

  This form updates both the working tree and the index from another commit/tree:

  ```
  git checkout main -- file.c
  ```

  That means: take file.c from main, put it into my working tree, and stage that version in the index.

  Recall that `--` signals the end of command options.

  The form

  ```
  git checkout -- file.c
  ```

  Restores the working-tree file from the index, so it affects the working directory, not the staging area.

- **Detached head**: Consider a commit with hash 5f1bc85

  ```
  git checkout 5f1bc85
  ```

  This checks out the commit whose hash starts with 5f1bc85.

  The important effect is that Git puts you in a detached HEAD state.

  Normally, HEAD points to a branch:

  ```
  HEAD -> main -> C
  ```

  After checking out a specific commit, HEAD points directly to that commit instead of to a branch:

  ```
  HEAD -> f51bc85
  ```

  So your working tree and index are updated to match the snapshot stored in commit 5f1bc85.

  You are not currently “on” a branch.

You can inspect old files, build the project, run tests, or look around safely.

But if you make a new commit from here:

```
git add .
git commit -m "experiment"
```

you get

```
A--B--C--D main
\
E HEAD
```

Commit E is not on a named branch unless you create one.

```
git switch -c experiment-branch
```

- **Git reset vs git checkout**:
  - **reset –hard:**
    1. moves the current branch
    2. resets index
    3. resets working tree
    4. discards tracked-file changes
  - **checkout:**
    1. moves HEAD to another commit
    2. leaves the branch where it is
    3. usually preserves/protects local changes, unless we pass -f
- **Delete untracked files**: Git clean removes untracked files from the working tree

```
git clean -fd # Force (f) and recursive (d)
```

- **git switch**: Switching branches updates the working tree and the index to match the branch being checked out

```
git switch main
git switch feature
```

So while git switch is not normally thought of as a staging command, it does replace the index as part of changing branches

- **git commit**: A plain commit reads the current index and creates a commit from it. The Git docs describe git commit as creating a new commit from the current contents of the index

```
git commit -a
```

Automatically stages modifications and deletions for already-tracked files before committing. It does not add brand-new untracked files.

```
git commit -p
```

Lets you interactively choose hunks to include in the commit.

```
git commit file.c
```

Commits the current content of the named tracked file, bypassing the normal "only what is already staged" behavior for that file.

- **git commit –amend**: Replaces the most recent commit with a new version of that commit.

  It is used when you want to modify the last commit instead of creating an additional commit.
- **git apply –index**: Applies a patch to both the working tree and the index.

```
git apply --index patch patch.diff
```

- **git apply –cached**: Applies a patch only to the index, leaving the working tree alone

```
git apply --cached patch.diff
```

  Without –index or –cached, git apply applies the patch only to files in the working tree.
- **git merge**: Merging updates the index and working tree when changes apply cleanly. If conflicts occur, the index stores special unmerged entries until you resolve them and stage the resolved files.

```
git merge feature
```

  After conflict resolution,

```
git add conflicted-file.c
git commit
```

- **git cherry-pick**: Applies the changes from an existing commit. If paths apply cleanly, they are updated in both the index and working tree. If conflicts occur, the index records multiple versions for conflicted paths.

```
git cherry-pick abc123
```

After resolving conflicts:

```
git add file.c
git cherry-pick --continue
```

- **git revert**: Creates a new commit that reverses an earlier commit. With –no-commit/-n, it applies the reverse changes to the working tree and index without committing.
- **git rebase**: Rebase repeatedly applies commits onto a new base. Internally, this affects the working tree and index as commits are replayed. If conflicts occur, you resolve files, stage them with git add, and continue:

```
git rebase main
git add file.c
git rebase --continue
```

- **Conflicts in a rebase**: After a git rebase, conflicts are handled one commit at a time. Git pauses at the first commit that cannot be replayed cleanly.

```
git status
```

Tells you which files have conflicts. Open each conflicted file and look for conflict markers.

```
<<<<<<< HEAD
code from the branch you are rebasing onto
=======
code from the commit currently being replayed
>>>>>>> commit-message-or-hash
```

Edit the file so it contains the final correct version. Remove the conflict markers. Then, stage the resolved files and continue the rebase

```
git rebase --continue
```

Git will then try to replay the next commit. If another conflict occurs, repeat the same process.

- **Rebase escape options**:
  1. **Abort**: Cancels the whole rebase and returns your branch to how it was before the rebase started.

```
git rebase --abort
```

2. **Skip**: Skips the commit that caused the conflict. Use this only when you are sure that commit's changes are no longer needed.

```
git rebase --skip
```

3. **Continue**: Continues after you resolve and stage the conflicts

```
git rebase --continue
```

– **Shortcut for resolving conflicts**: git checkout –ours file and git checkout –theirs file are shortcuts for resolving a conflicted file by choosing one side completely.

They are used when Git has stopped because of a conflict.

```
git checkout --ours file.cpp
```

Means replace file.cpp with our side of the conflict.

```
git checkout --theirs file.cpp
```

Means replace file.cpp with their side of the conflict. After choosing one, you still need to stage the file:

```
git add file.cpp
```

Then, continue the operation

```
git merge --continue
```

However, during a rebase the meaning feels reversed. Suppose we are replaying feature commits on top of main. So, we are on branch `feature` and ran

```
git rebase main
```

In this case,

```
git checkout --ours file.cpp
```

chooses the version from main, the branch we are rebasing onto.

```
git checkout --theirs file.cpp
```

chooses the version from the `feature` commit currently being replayed.

Note: Modern git also lets you write this as

```
git restore --ours file.cpp
git restore --theirs file.cpp
```

– **Interactive rebasing**: git rebase -i means interactive rebase. It lets you rewrite a sequence of commits by editing a temporary "todo list" before Git replays those commits.

```
git rebase -i main
```

Git opens a list like this:

```
pick a1b2c3d Add parser
pick b2c3d4e Fix parser bug
pick c3d4e5f Add tests
pick d4e5f6a Rename parse function
```

You can change **pick**, reorder lines, delete lines, or combine commits.

Suppose your history is:

```
A -- B -- C -- D   feature
```

Maybe B, C, and D are really one logical change.

```
B: Add parser
C: Fix typo in parser
D: Fix parser edge case
```

With interactive rebase, you can squash them:

```
pick B Add parser
squash C Fix typo in parser
squash D Fix parser edge case
```

After the rebase, Git creates one new commit containing the combined changes:

```
A -- B'
```

Because Git gives you a list of commits, you can literally move lines around. Suppose the original order is

```
pick B Add UI
pick C Add database schema
pick D Connect UI to database
```

You might change it to

```
pick C Add database schema
pick B Add UI
pick D Connect UI to database
```

This makes Git replay the database commit first, then the UI commit, then the connection commit.

This is useful when the original order was accidental or confusing. However, the reordered commits still need to make sense. If commit D depends on code from B, putting D before B may cause conflicts or broken intermediate commits.

We can remove a commit from the branch history by replacing **pick** with **drop** or simply deleting the line. This is different from git revert. A revert creates a new commit that undoes an old one. Interactive rebase rewrites history as if the bad commit was never there.

– **More specific rebasing**: You can also use –onto for more specific rebasing:

```
rebase --onto new_base old_base branch
```

Take the commits after old_base on branch, and replay them onto new_base. That lets you move a branch onto almost any commit in the repository.

– **git pull**: git pull is essentially git fetch followed by either git merge or git rebase, depending on configuration/options. Because the merge/rebase part can update the index, git pull can affect the staging area indirectly.

```
git pull
git pull --rebase
```

– **git stash**: git stash records the current state of both the working directory and the index, then reverts the working directory to match HEAD.

```
git stash

git stash --staged
git stash --keep-index
git stash pop
git stash apply
```

git stash pop and git stash apply can also modify the index depending on how the stash was created and applied.

- **Low-level index manipulations**:
  - **git update-index**: This is the direct plumbing command for modifying the index. It can add entries, remove entries, refresh stat information, change executable bits, set assume-unchanged, set skip-worktree, and more. Git's own documentation says git update-index "modifies the index."

```
git update-index --chmod=+x script.sh
git update-index --assume-unchanged config.local
git update-index --skip-worktree config.local
```

  - **git read-tree**: Low-level command for reading tree objects into the index. It is used internally by commands such as checkout, merge, and reset-like operations.

```
git read-tree HEAD
```

- **git ls-files**: git ls-files lists files that Git knows about in the index/staging area, and optionally files in the working tree

```
git ls-files
```

This prints the files that are currently tracked by Git, because tracked files are recorded in the index.

```
git ls-files --others
```

Shows untracked files

```
git ls-files --others --exclude-standard
```

Shows untracked files, but respect .gitignore.

```
git ls-files --ignored --others --exclude-standard
```

```
git ls-files --deleted
```

Shows deleted tracked files

```
git ls-files --modified
```

Shows modified tracked files

```
git ls-files --unmerged
```

Shows files with merge conflicts.

- **git status**: Shows the current state of your repo relative to the current commit, usually HEAD.
- **git log**: Shows the commit history

```
git log
```

Show recent commits only:

```
git log -5
```

Show commits affecting a specific file:

```
git log -- main.c
```

Show patches introduced by each commit:

```
git log -p
```

Show summary of changed files:

```
git log --stat
```

- **Tracking**: In Git, to **track** means: A local branch is linked to a remote branch so Git knows which remote branch it should compare, pull from, and push to by default.
- **Upstreams**: An upstream is the remote branch that your local branch is configured to "track."

For example, suppose you have:

```
main
origin/main
```

If your local `main` is tracking `origin/main`, then origin/main is the upstream of main. You can see this with

```
git branch -vv
```

```
* main  abc1234 [origin/main] Add parser notes
```

So, local branch main has upstream origin/main. The upstream matters because commands like these use it automatically.

```
git pull
git push
git status
```

If your branch has an upstream, then

```
git push
```

usually means

```
git push origin main
```

Also,

```
git pull
```

usually means

```
git fetch origin
git merge origin/main
```

- **Setting an upstream**: When you first push a local branch, you often use

```
git push -u origin main
```

Or, equivalently

```
git push --set-upstream origin main
```

The -u means *set upstream*.

You can also manually set an upstream:

```
git branch --set-upstream-to=origin/main main
```

Or, if already on main,

```
git branch --set-upstream-to=origin/main
```

- **Upstreams and branches**: You do not always need to set an upstream for a branch.

  You only need an upstream if you want Git to know which remote branch your local branch corresponds to for commands like:

  ```
  git pull
  git push
  git status
  ```

  Suppose you create a local branch

  ```
  git switch -c feature
  ```

  At this point, feature is just local. It does not automatically have an upstream unless you created it from a remote branch in a tracking-aware way.

  If you only want to work locally, no upstream is needed.

  But if you want to push it, you usually do

  ```
  git push -u origin feature
  ```

  This does two things

  1. Creates / updates the remote branch

     ```
     origin/feature
     ```

  2. Sets your local branch **feature** to track **origin/feature**.

- **Remote-tracking branch**: A remote-tracking branch is Gits local record of a branch on a remote.

```
origin/main
origin/dev
origin/feature-x
```

These are not branches you usually edit directly. They are Git's local snapshots of what the remote branches looked like the last time you fetched.

An **upstream** is a relationship between your local branch and one of those remote-tracking branches.

- **Rebase**: A rebase moves your commits so that they appear to be based on a different commit. Suppose we have history

```
A---B---C  origin/main
\
D---E  feature
```

You made commits D and E on feature, but meanwhile main advanced to C. If you run

```
git switch feature
git rebase origin/main
```

Git rewrites your branch like this:

```
A---B---C  origin/main
\
D'---E'  feature
```

Your changes are replayed on top of the latest origin/main.

Notice the commits are now called D' and E', not D and E. That is because rebasing creates new commits with new commit hashes.

- **Rebase vs merge**: Both merge and rebase can bring one branch up to date with another branch, but they do it differently.
  - **Merge**:

```
git merge origin/main
```

creates a new merge commit:

```
A---B----C  origin/main
\     \
D---E--M  feature
```

The history preserves the fact that the branches diverged and were merged.

- **Rebase**:

```
git rebase origin/main
```

rewrites the feature branch

```
A---B---C  origin/main
\
D'---E'  feature
```

The history looks linear, as if you started your feature branch from the latest main. Rebase is commonly used to keep a clean, linear history.

For example, before merging a feature branch, you might do:

```
git fetch origin
git rebase origin/main
```

This updates your feature branch so your commits are placed after the newest commits on main.

- **Rebasing with upstream**: If your branch has an upstream, you can do:

```
git pull --rebase
```

This means:

```
git fetch
git rebase <upstream>
```

So instead of fetching and merging the upstream branch, Git fetches and rebases your local commits on top of it.

- **Blobs**: a blob is the object type that stores the contents of a file.

A blob does not store the file name, path, permissions, or directory structure. It only stores the raw file data.

If you have

```
src/main.c
```

Git stores the contents of main.c as a **blob**. The filename main.c and the path src/main.c are stored separately in a **tree object**.

- **blob**: file contents
- **tree**: directory listing: names + permissions + pointers to blobs/trees
- **commit**: snapshot metadata + pointer to a tree

- **git gc and git prune**:

```
git gc
```

Means **Git garbage collection**. It is a maintenance command that cleans and optimizes the internal .git database. It does several things, including:

1. **Compresses loose objects into packfiles**: When Git creates objects, such as blobs, trees, commits, and tags, they may initially be stored as individual files under:

   ```
   .git/objects/
   ```

   These are called loose objects. Over time, Git packs many objects together into compressed files called packfiles

   ```
   .git/objects/pack/
   ```

   This saves disk space and makes many operations faster.
2. **Removes unreachable objects, but conservatively**: Objects that are not reachable from any branch, tag, stash, reflog, or other reference may eventually be deleted.

   ```
   git add file.txt
   git reset file.txt
   ```

   Git may have created a blob during git add, but after reset, nothing points to that blob anymore. That blob is now unreachable.

   git gc may eventually remove it, but Git is conservative. It usually keeps recent unreachable objects around for a while in case you need to recover them.
3. **Cleans up other internal metadata**: It may also prune old reflog entries, repack objects, clean redundant packfiles, and optimize internal repository storage.

`git prune` is more specific. It removes **unreachable loose objects** from the object database

```
git prune
```

An object is unreachable if no reference path leads to it. For example, if no branch, tag, commit, index entry, stash, or reflog points to it, Git may consider it garbage.

- **git show**: used to view detailed metadata and textual differences (diffs) for specific Git objects, most commonly commits. By default, running it without any arguments displays the author, date, log message, and code changes of the absolute latest commit (HEAD) in your current branch

```
git show
```

We can also inspect a specific commit

```
git show <commit-hash>
```

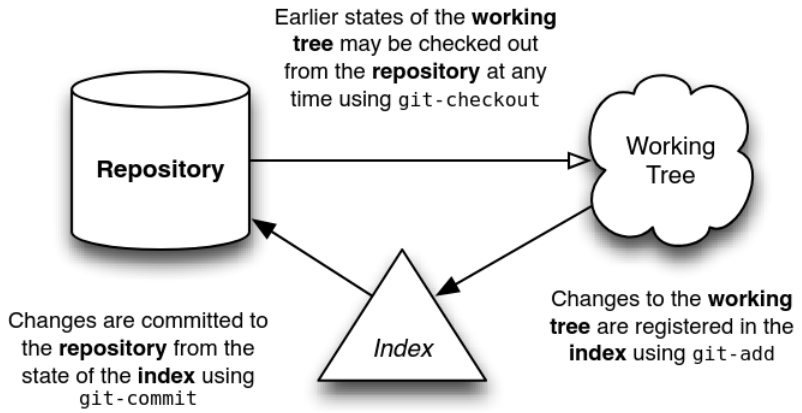
Displays metadata alongside a complete line-by-line breakdown of code additions and deletions.

- **Inspection differences**:

Command	Primary Scope	Default Focus
git show	A single object/commit.	Displays complete metadata and the full code diff.
git log	A chronological list or range of commits.	Displays only structural metadata like hashes, authors, and log messages.
git diff	A comparison between two states.	Displays differences between your working files, the staging area, or separate branches.

- **Summary**:
  - **repository**: A repository is a collection of commits, each of which is an archive of what the project's working tree looked like at a past date, whether on your machine or someone else's. It also defines HEAD (see below), which identifies the branch or commit the current working tree stemmed from. Lastly, it contains a set of branches and tags, to identify certain commits by name.
  - **index**: Unlike other, similar tools you may have used, Git does not commit changes directly from the working tree into the repository. Instead, changes are first registered in something called the index. Think of it as a way of "confirming" your changes, one by one, before doing a commit (which records all your approved changes at once). Some find it helpful to call it instead as the "staging area", instead of the index.
  - **working tree**: A working tree is any directory on your filesystem which has a repository associated with it (typically indicated by the presence of a sub-directory within it named .git.). It includes all the files and sub-directories in that directory.

– **commit**: A commit is a snapshot of your working tree at some point in time. The state of HEAD (see below) at the time your commit is made becomes that commit's parent. This is what creates the notion of a "revision history".
– **branch**: A branch is just a name for a commit (and much more will be said about commits in a moment), also called a reference. It's the parentage of a commit which defines its history, and thus the typical notion of a "branch of development".
– **tag**: A tag is also a name for a commit, similar to a branch, except that it always names the same commit, and can have its own description text.
– **master**: The mainline of development in most repositories is done on a branch called "master". Although this is a typical default, it is in no way special.
– **HEAD**: HEAD is used by your repository to define what is currently checked out:
  * If you checkout a branch, HEAD symbolically refers to that branch, indicating that the branch name should be updated after the next commit operation.
  * If you checkout a specific commit, HEAD refers to that commit only. This is referred to as a detached HEAD, and occurs, for example, if you check out a tag name.



Blobs and trees

- **Data is immutable**: Once Git stores an object in the repository, Git does not modify that object in place. Instead, if something changes, Git creates a **new object** with a **new hash**.

  Git's object database stores objects such as:

  - **blob** : file contents
  - **tree** : directory structure
  - **commit** : snapshot metadata + pointer to a tree
  - **tag** : annotated tag object

  Each object is identified by a hash of its contents. For example, a blob's ID is computed from the blob's data plus a small header.

  $$\text{hash(contents)} = \text{object ID}.$$

  So, if the contents change, the hash changes.

  ```
  1  echo "hello" | git hash-object --stdin
  2  echo "hello world" | git hash-object --stdin
  ```

  Those produce different object IDs because the contents are different. That means Git cannot just "edit" an existing blob. If it did, the object's contents would no longer match its hash. The old object ID would become invalid.

  Git does this

  ```
  1  blob abc123: "hello"
  2  blob def456: "hello world"
  ```

  The old object still exists. The new version is stored as a separate object. The same idea applies to commits. A commit contains data like:

  ```
  1  tree pointer
  2  parent commit pointer
  3  author
  4  committer
  5  message
  6  timestamp
  ```

  If any of that changes, the commit hash changes. So when you "amend" a commit, Git does not actually edit the old commit. It creates a new commit object.

- **Blobs and trees**: A blob represents the contents of one file at one particular version.

A **Git tree** is basically Git's representation of a directory. A tree object contains entries. Each entry records

mode name object id

For files, the object ID points to a blob

100644 main.c blob abc123 100644 README.md blob def456

For subdirectories, the object ID points to another **tree**.

- **Object modes**: In Git, object modes are the mode values stored in tree objects to describe what kind of entry each path is.

Mode	Meaning
100644	Regular file, not executable
100755	Regular file, executable
040000	Directory, represented by a tree object
120000	Symbolic link
160000	Git submodule, also called a gitlink

- **Intro**: What Git does is quite rudimentary: it maintains snapshots of a directory's contents. Much of its internal design can be understood in terms of this basic task.

  The design of a Git repository in many ways mirrors the structure of a UNIX filesystem: A filesystem begins with a root directory, which typically consists of other directories, most of which have leaf nodes, or files, that contain data. Meta-data about these files' contents is stored both in the directory (the names), and in the i-nodes that reference the contents of those file (their size, type, permissions, etc). Each i-node has a unique number that identifies the contents of its related file. And while you may have many directory entries pointing to a particular i-node (i.e., hard-links), it's the i-node which "owns" the contents stored on your filesystem.

  Internally, Git shares a strikingly similar structure, albeit with one or two key differences. First, it represents your file's contents in blobs, which are also leaf nodes in something awfully close to a directory, called a tree. Just as an i-node is uniquely identified by a system-assigned number, a blob is named by computing the SHA1 hash id of its size and contents. For all intents and purposes this is just an arbitrary number, like an i-node, except that it has two additional properties: first, it verifies the blob's contents will never change; and second, the same contents shall always be represented by the same blob, no matter where it appears: across commits, across repositories — even across the whole Internet. If multiple trees reference the same blob, this is just like hard-linking: the blob will not disappear from your repository as long as there is at least one link remaining to it.

  The difference between a Git blob and a filesystem's file is that a blob stores no metadata about its content. All such information is kept in the tree that holds the blob. One tree may know those contents as a file named "foo" that was created in August 2004, while another tree may know the same contents as a file named "bar" that

was created five years later. In a normal filesystem, two files with the same contents but with such different metadata would always be represented as two independent files. Why this difference? Mainly, it's because a filesystem is designed to support files that change, whereas Git is not. The fact that data is immutable in the Git repository is what makes all of this work and so a different design was needed. And as it turns out, this design allows for much more compact storage, since all objects having identical content can be shared, no matter where they are.

Note: Git tracks commits. A commit points to a tree, the tree maps filenames to blobs, and the blobs store file contents.

- **The blob**: We can create a simple example,

```
mkdir sample && cd sample && echo "Hello, world!" > greeting
git hash-object greeting # ->
  ↪  af5626b4a114abcb82d63db7c8082c3c4756e51b
```

No matter where you run `git hash-object`, if the file has only the content *Hello, world!*, the hash will be the same.

If another repository has a commit whose tree refers to the same blob object, then when we fetch or pull from that repository, Git can recognize that we already have that blob because its object ID is the same. Git only needs to download/store objects that we do not already have.

If we now create a repository,

```
git init
git add greeting
git commit -m "Added greeting"
```

Our blob should be in the system exactly as we expected, using the hash id de-termined above. As a convenience, Git requires only as many digits of the hash id as are necessary to uniquely identify it within the repository. Usually just six or seven digits is enough:

```
git cat-file -t af5626b # -> blob
git cat-file blob af5626b # -> Hello, world!
```

a Git blob represents the fundamental data unit in Git. Really, the whole system is about blob management.

- **Blobs are stored in trees** In order for Git to represent the structure and naming of your files, it attaches blobs as leaf nodes within a tree. Now, I can't discover which tree(s) a blob lives in just by looking at it, since it may have many, many owners. But I know it must live somewhere within the tree held by the commit I just made:

```
git ls-tree HEAD # -> 100644 blob
↪  af5626b4a114abcb82d63db7c8082c3c4756e51b greeting
```

This commit contains one Git tree, which has a single leaf: the greeting content's blob.

Although I can look at the tree containing my blob by passing HEAD to ls-tree, I haven't yet seen the underlying tree object referenced by that commit. Here are a few other commands to highlight that difference and thus discover my tree:

```
git rev-parse HEAD # ->
↪  0437733255614428faae0e090712264cf336cd37
git cat-file -t HEAD # -> commit
git cat-file commit HEAD

tree 0563f77d884e4f79ce95117e2d686d7d6e282887
author hgoose <noodlecloudcode@gmail.com> 1780326086 -0500
committer hgoose <noodlecloudcode@gmail.com> 1780326086 -0500

b # -> commit message
```

The first command decodes the HEAD alias into the commit it references, the second verifies its type, while the third command shows the hash id of the tree held by that commit, as well as the other information stored in the commit object. The hash id for the commit is unique to my repository — because it includes my name and the date when I made the commit — but the hash id for the tree should be common between your example and mine, containing as it does the same blob under the same name.

```
git ls-tree 0563f77 # -> 100644 blob
↪  af5626b4a114abcb82d63db7c8082c3c4756e51b greeting
```

There you have it: my repository contains a single commit, which references a tree that holds a blob — the blob containing the contents I want to record. There's one more command I can run to verify that this is indeed the case:

```
find .git/objects -type f | sort
.git/objects/04/37733255614428faae0e090712264cf336cd37
.git/objects/05/63f77d884e4f79ce95117e2d686d7d6e282887
.git/objects/af/5626b4a114abcb82d63db7c8082c3c4756e51b
```

We see three objects. These objects should refer to a tree, a commit, and a blob. Let's take a look

```
git cat-file -t 0437733255
commit
git cat-file -t 0563f77
tree
git cat-file -t af5626b
blob
```

Notice the directory name is the MSB of each hash.

```
git show 0437733
commit 0437733255614428faae0e090712264cf336cd37 (HEAD ->
↪  main)
Author: hgoose <noodlecloudcode@gmail.com>
Date:   Mon Jun 1 10:01:26 2026 -0500

b

diff --git a/greeting b/greeting
new file mode 100644
index 0000000..af5626b
--- /dev/null
+++ b/greeting
@@ -0,0 +1 @@
+Hello, world!

git show 0563f77
tree 0563f77

greeting

git show af5626b
Hello, world!
```

- **How trees are made**: Every commit holds a single tree, but how are trees made? We know that blobs are created by stuffing the contents of your files into blobs — and that trees own blobs — but we haven't yet seen how the tree that holds the blob is made, or how that tree gets linked to its parent commit.

  Let's start with a new sample repository again, but this time by doing things manually, so you can get a feeling for exactly what's happening under the hood:

```
rm -rf greeting .git
echo "Hello, world!" > greeting
git init
git add greeting
```

It all starts when you first add a file to the index.

```
git log # Fails, no commits

git ls-files --stage
100644 af5626b4a114abcb82d63db7c8082c3c4756e51b 0 greeting
```

What's this? I haven't committed anything to the repository yet, but already an object has come into being. It has the same hash id I started this whole business with, so I know it represents the contents of my greeting file. I could use cat-file -t at this point on the hash id, and I'd see that it was a blob. It is, in fact, the same blob I got the first time I created this sample repository.

This blob isn't referenced by a tree yet, nor are there any commits. At the moment it is only referenced from a file named .git/index, which references the blobs and trees that make up the current index. So now let's make a tree in the repo for our blob to hang off of:

```
git write-tree # Records the contents of the index in a tree
0563f77d884e4f79ce95117e2d686d7d6e282887
```

This number should look familiar as well: a tree containing the same blobs (and subtrees) will always have the same hash id. I don't have a commit object yet, but now there is a tree object in that repository which holds the blob. The purpose of the low-level write-tree command is to take whatever the contents of the index are and tuck them into a new tree for the purpose of creating a commit.

I can manually make a new commit object by using this tree directly, which is just what the commit-tree command does:

```
git commit-tree 0563f77 -m "Initial commit"
d63d26194dd7a5cfbb9fe7365874c2918707f091
```

The raw commit-tree command takes a tree's hash id and makes a commit object to hold it. If I had wanted the commit to have a parent, I would have had to specify the parent commit's hash id explicitly using the -p option.

Our work is not done yet, though, since I haven't registered the commit as the new head of the current branch:

```
echo d63d26194dd7a5cfbb9fe7365874c2918707f091 >
↪  .git/refs/head/master
```

This command tells Git that the branch name "master" should now refer to our recent com- mit. Another, much safer way to do this is by using the command update-ref

```
git update-ref refs/heads/master d63d2619
```

After creating master, we must associate our working tree with it. Normally this happens for you whenever you check out a branch.

```
git symbolic-ref HEAD refs/heads/master
```

This command associates HEAD symbolically with the master branch. This is significant be- cause any future commits from the working tree will now automatically update the value of refs/heads/master.

The command git symbolic-ref HEAD refs/heads/master manually forces Git's HEAD pointer to point directly to the master branch without altering your working directory or changing your files. It writes the string ref: refs/heads/master straight into .git/HEAD.

It's hard to believe it's this simple, but yes, I can now use log to view my newly minted commit

```
git log
```

A side note: if I hadn't set refs/heads/master to point to the new commit, it would have been considered "unreachable", since nothing currently refers to it nor is it the parent of a reach- able commit. When this is the case, the commit object will at some point be removed from the repository, along with its tree and all its blobs. (This happens automatically by a command called gc, which you rarely need to use manually). By linking the commit to a name within refs/ heads, as we did above, it becomes a reachable commit, which ensures that it's kept around from now on.

Commits

- **The beauty of commits**: Some version control systems make "branches" into magical things, often distinguishing them from the "main line" or "trunk", while others discuss the concept as though it were very different from commits. But in Git there are no branches as separate entities: there are only blobs, trees and commits. Since a commit can have one or more parents, and those commits can have parents, this is what allows a single commit to be treated like a branch: because it knows the whole history that led up to it.

  You can examine all the top-level, referenced commits at any time using the branch command:

  ```
  1  git branch -v
  ```

  Say it with me: A branch is nothing more than a named reference to a commit. In this way, branches and tags are identical, with the sole exception that tags can have their own descriptions, just like the commits they reference. Branches are just names, but tags are descriptive, well, "tags".

  But the fact is, we don't really need to use aliases at all. For example, if I wanted to, I could reference everything in the repository using only the hash ids of its commits. Here's me being straight up loco and resetting the head of my working tree to a particular commit:

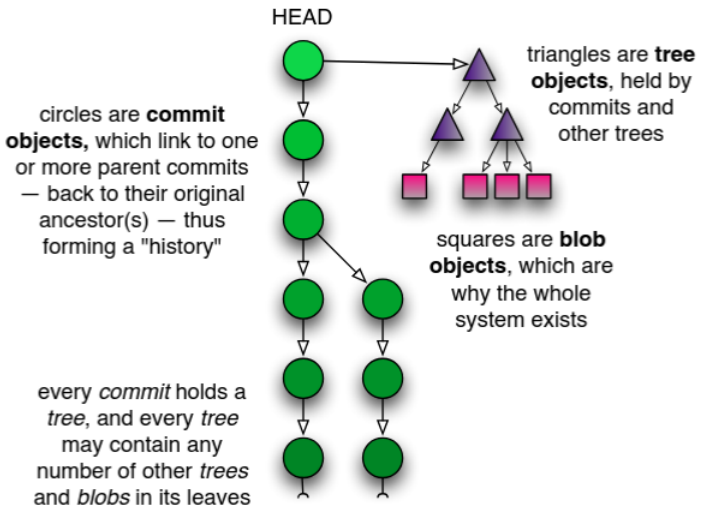
  ```
  1  git reset --hard 5f1bc85
  ```

  The –hard option says to erase all changes currently in my working tree, whether they've been registered for a checkin or not (more will be said about this command later). A safer way to do the same thing is by using checkout:

  ```
  1  git checkout 5f1bc85
  ```

  The difference here is that changed files in my working tree are preserved. If I pass the -f option to checkout, it acts the same in this case to reset –hard, except that checkout only ever changes the working tree, whereas reset –hard changes the current branch's HEAD to ref- erence the specified version of the tree.

  Another joy of the commit-based system is that you can rephrase even the most complicated version control terminology using a single vocabulary. For example, if a commit has multiple parents, it's a "merge commit" — since it merged multiple commits into one. Or, if a commit has multiple children, it represents the ancestor of a "branch", etc. But really there is no difference between these things to Git: to it, the world is simply a collection of commit objects, each of which holds a tree that references other trees and blobs, which store your data. Anything more complicated than this is simply a device of nomenclature.



- **A commit by any other name**: If commits are the key, how you name commits is the doorway to mastery. There are many, many ways to name commits, ranges of commits, and even some of the objects held by commits, which are accepted by most of the Git commands. Here's a summary of some of the more basic usages:
  - **branchname**: As has been said before, the name of any branch is simply an alias for the most recent commit on that "branch". This is the same as using the word HEAD whenever that branch is checked out.
  - **tagname**: A tag-name alias is identical to a branch alias in terms of naming a commit. The major difference between the two is that tag alias- es never change, whereas branch aliases change each time a new commit is checked in to that branch.
  - **HEAD**: The currently checked out commit is always called HEAD. If you check out a specific commit — instead of a branch name — then HEAD refers to that commit only and not to any branch. Note that this case is somewhat special, and is called "using a detached HEAD" (I'm sure there's a joke to be told here…).
  - **c82a22c39cbc32**: A commit may always be referenced using its full, 40-character SHA1 hash id. Usually this happens during cut-and-pasting, since there are typically other, more convenient ways to refer to the same commit.
  - **c82a22c**: You only need use as many digits of a hash id as are needed for a unique reference within the repository. Most of the time, six or seven digits is enough.
  - **name**∧: The parent of any commit is referenced using the caret symbol. If a commit has more than one parent, the first is used.
  - **name**∧∧: Carets may be applied successively. This alias refers to "the parent of the parent" of the given commit name.
  - **name**∧2: If a commit has multiple parents (such as a merge commit), you can refer to the nth parent using name∧n.
  - **name**∼ 10: A commit's nth ancestor may be referenced using a tilde (∼) followed by the ordinal number. This type of usage is common with `rebase -i`, for example, to mean "show me a bunch of recent commits". This is the same as name∧ ∧ ∧ ∧ ∧ ∧ ∧ ∧ ∧∧.

- **name:path:** To reference a certain file within a commit's content tree, specify that file's name after a colon. This is helpful with show, or to show the difference between two versions of a committed file:

```
git diff HEAD^1:Makefile HEAD^2:Makefile
```

- **name^{tree}**: You can reference just the tree held by a commit, rather than the commit itself.
- **name1..name2**: This and the following aliases indicate commit ranges, which are supremely useful with commands like log for seeing what's happened during a particular span of time.

  The syntax to the left refers to all the commits reachable from name2 back to, but not including, name1. If either name1 or name2 is omitted, HEAD is used in its place.
- **name1...name2**: A "triple-dot" range is quite different from the two-dot version above. For commands like log, it refers to all the commits refer- enced by name1 or name2, but not by both. The result is then a list of all the unique commits in both branches.

  For commands like diff, the range expressed is between name2 and the common ancestor of name1 and name2. This differs from the log case in that changes introduced by name1 are not shown.
- **master..**: This usage is equivalent to "master..HEAD". I'm adding it here, even though it's been implied above, because I use this kind of alias constantly when reviewing changes made to the current branch.
- **..master**: This, too, is especially useful after you've done a fetch and you want to see what changes have occurred since your last rebase or merge.
- **–since="2 weeks ago"**: Refers to all commits since a certain date.
- **–until="1 week ago"**: Refers to all commits up to a certain date.
- **–grep=pattern**: Refers to all commits whose commit message matches the regular expression pattern.
- **–committer=pattern**: Refers to all commits whose committer matches the pattern.
- **–author=pattern**: Refers to all commits whose author matches the pattern. The author of a commit is the one who created the changes it represents. For local development this is always the same as the committer, but when patches are being sent by e-mail, the author and the committer usually differ.
- **–no-merges**: Refers to all commits in the range that have only one parent — that is, it ignores all merge commits.

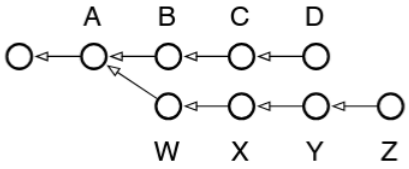
Most of these options can be mixed-and-matched. Here is an example which shows the fol- lowing log entries: changes made to the current branch (branched from master), by myself, within the last month, which contain the text "foo":

```
git log --grep="foo" --author="johnw" --since="1 month ago"
↪ master..
```

Branching, merging, and rebasing

- **Branching and the power of rebase**: One of Git's most capable commands for manipulating commits is the innocently-named re- base command. Basically, every branch you work from has one or more "base commits": the commits that branch was born from.

  Take the following typical scenario, for example. Note that the arrows point back in time because each commit references its parent(s), but not its children. Therefore, the D and Z commits represent the heads of their respective branches:



  In this case, running branch would show two "heads": D and Z, with the common parent of both branches being A. The output of show-branch shows us just this information:

```
git branch
Z
* D
```

```
git show-branch
! [Z] Z
* [D] D
--
* [D] D
* [D^] C
* [D~2] B
+ [Z] Z
+ [Z^] Y
+ [Z~2] X
+ [Z~3] W
+* [D~3] A
```

  Reading this output takes a little getting used to, but essentially it's no different from the diagram above.

  - The branch we're on experienced its first divergence at commit A (also known as commit $D \sim 3$, and even $Z \sim 4$ if you feel so inclined).
  - Reading from bottom to top, the first column (the plus signs) shows a divergent branch named Z with four commits: W, X, Y and Z.
  - The second column (the asterisks) show the commits which happened on the current branch, namely three commits: B, C and D.

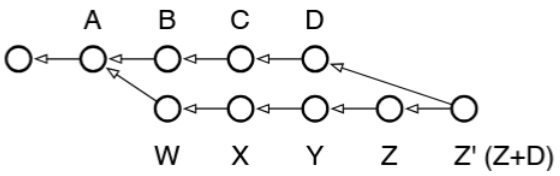
– The top of the output, separated from the bottom by a dividing line, identifies the branches displayed, which column their commits are labelled by, and the character used for the labeling.

The action we'd like to perform is to bring the working branch Z back up to speed with the main branch, D. In other words, we want to incorporate the work from B, C, and D into Z

In other version control systems this sort of thing can only be done using a "branch merge". In fact, a branch merge can still be done in Git, using merge, and remains needful in the case where Z is a published branch and we don't want to alter its commit history.

```
1 git checkout Z
2 git merge D
```

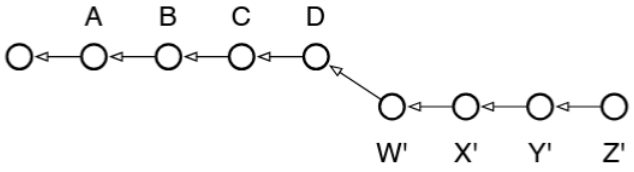
This is what the repository looks like afterward:



If we checked out the Z branch now, it would contain the contents of the previous Z (now referenceable as $Z\wedge$), merged with the contents of D. (Though note: a real merge operation would have required resolving any conflicts between the states of D and Z).

Although the new Z now contains the changes from D, it also includes a new commit to represent the merging of Z with D: the commit now shown as Z'. This commit doesn't add anything new, but represents the work done to bring D and Z together. In a sense it's a "meta-commit", because its contents are related to work done solely in the repository, and not to new work done in the working tree.

There is a way, however, to transplant the Z branch straight onto D, effectively moving it for- ward in time: by using the powerful rebase command. Here's the graph we're aiming for:



This state of affairs most directly represents what we'd like done: for our local, development branch Z to be based on the latest work in the main branch D. That's why the command is called "rebase", because it changes the base commit of the branch it's run from. If you run it repeatedly, you can carry forward a set of patches indefinitely, always staying up-to-date with the main branch, but without adding unnecessary merge commits to your development branch

```
git checkout Z
git rebase D # Change Z's base commit to point to D
```

Why is this only for local branches? Because every time you rebase, you're potentially changing every commit in the branch. Earlier, when W was based on A, it contained only the changes needed to transform A into W. After running rebase, however, W will be rewritten to contain the changes necessary to transform D into W'. Even the transformation from W to X is changed, because $A + W + X$ is now $D + W' + X'$ - and so on. If this were a branch whose changes are seen by other people, and any of your downstream consumers had created their own local branches off of Z, their branches would now point to the old Z, not the new Z'

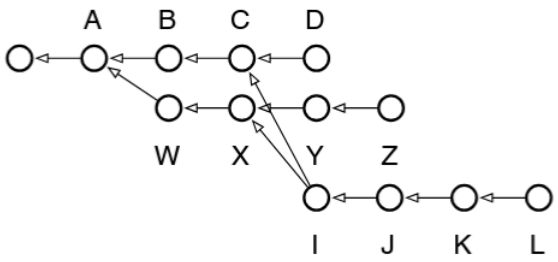
Generally, the following rule of thumb can be used: Use rebase if you have a local branch with no other branches that have branched off from it, and use merge for all other cases. Merge is also useful when you're ready to pull your local branch's changes back into the main branch.

- **Interactive rebasing**: When rebase was run above, it automatically rewrote all the commits from W to Z in order to rebase the Z branch onto the D commit (i.e., the head commit of the D branch). You can, how- ever, take complete control over how this rewriting is done. If you supply the -i option to rebase, it will pop you into an editing buffer where you can choose what should be done for every commit in the local Z branch:
  - **pick** This is the default behavior chosen for every commit in the branch if you don't use interactive mode. It means that the commit in question should be applied to its (now rewritten) parent commit. For every commit that involves conflicts, the rebase command gives you an opportunity to re- solve them.
  - **squash**: A squashed commit will have its contents "folded" into the contents of the commit preceding it. This can be done any number of times. If you took the example branch above and squashed all of its commits (except the first, which must be a pick in order to squash), you would end up with a new Z branch containing only one commit on top of D. Useful if you have changes spread over multiple commits, but you'd like the history rewritten to show them all as a single commit.
  - **edit**: If you mark a commit as edit, the rebasing process will stop at that commit and leave you at the shell with the current working tree set to reflect that commit. The index will have all the commit's changes registered for inclusion when you run commit. You can thus make whatever changes you like: amend a change, undo a change, etc.; and after committing, and running rebase –continue, the commit will be rewritten as if those changes had been made originally.
  - **(drop)**: If you remove a commit from the interactive rebase file, or if you comment it out, the commit will simply disappear as if it had never been checked in. Note that this can cause merge conflicts if any of the later commits in the branch depended on those changes.

The power of this command is hard to appreciate at first, but it grants you virtually unlimited control over the shape of any branch. You can use it to:

  - Collapse multiple commits into single ones.
  - Re-order commits.
  - Remove incorrect changes you now regret.
  - Move the base of your branch onto any other commit in the repository.
  - Modify a single commit, to amend a change long after the fact.

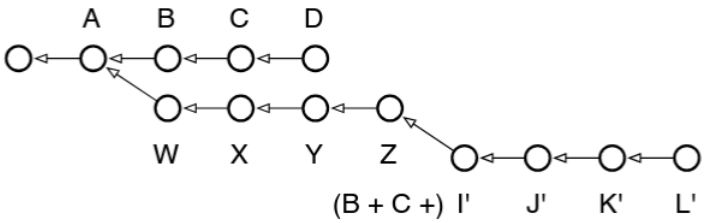
- **Rebase example**: Consider the history



Suppose we wanted to migrate the secondary branch L to become the new head of Z. We simply

```
git checkout L
git rebase Z
```

Now, we have



As you can see, when it comes to local development, rebasing gives you unlimited control over how your commits appear in the repository.

- **Rebase only with local development**: Rebasing is recommended mainly for local development because it rewrites commit history.

  A rebase does not move the same commits. It creates new commits with new parent commits, so they get new hashes.

  Suppose we have the history

```
A -- B -- C          main
\
D -- E        feature
```

If we run

```
git checkout feature
git rebase main
```

Git replays D and E on top of C.

```
A -- B -- C -- D' -- E'    feature
```

The commits D' and E' contain similar changes to D and E, but they are not the same commits. Their parent changed, so their hashes changed.

That is safe if only your local repository knows about D and E.

The problem happens when other people already have the old commits.

Suppose we push

```
A -- B -- C          main
\
D -- E        feature
```

Then, someone else creates work on top of E

```
A -- B -- C          main
\
D -- E -- F   coworker-work
```

Now, you rebase and force-push

```
A -- B -- C -- D' -- E'   feature
```

Their branch is still based on the old E:

```
A -- B -- C -- D' -- E'    feature
\
D -- E -- F         coworker-work
```

Now the project has two versions of the same logical changes:

```
D, E      old commits
D', E'    rebased replacements
```

So, rebase freely before you push. Avoid rebasing commits that other people may already have.

Rebase unpublished/local history. Do not casually rebase published/shared history.

You can always undo the effect of a merge commit with

```
git revert -m 1 M
```

The -m 1 means: Treat parent 1 as the mainline parent, usually the branch you were on when you merged.

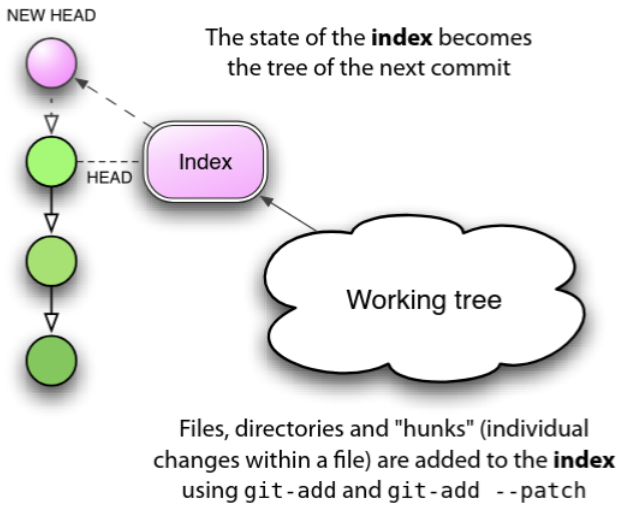
Merging is safe because it preserves existing history. Being able to revert the merge commit is a useful recovery mechanism, but the core safety property is that merge is additive rather than history-rewriting.

The Index

- **Intro**: Between your data files, which are stored on the filesystem, and your Git blobs, which are stored in the repository, there stands a somewhat strange entity: the Git index. Part of what makes this beast hard to understand is that it's got a rather unfortunate name. It's an index in the sense that it refers to the set of newly created trees and blobs which you created by running add. These new objects will soon get bound into a new tree for the purpose of committing to your repository — but until then, they are only referenced by the index. That means that if you unregister a change from the index with reset, you'll end up with an orphaned blob that will get deleted at some point at the future.

  The index is really just a staging area for your next commit, and there's a good reason why it exists: it supports a model of development that may be foreign to users of CVS or Subversion, but which is all too familiar to Darcs users: the ability to build up your next commit in stages

  git add creates blob objects and records them in the index. Later, when committing, Git writes tree objects based on the index.



  First, let me say that there is a way to ignore the index almost entirely: by passing the -a flag to commit. Look at the way Subversion works, for example. When you type svn status, what you'll see is a list of actions to be applied to your repository on the next call to svn commit. In a way, this "list of next actions" is a kind of informal index, determined by comparing the state of your working tree with the state of HEAD. If the file foo.c has been changed, on your next commit those changes will be saved. If an unknown file has a question mark next to it, it will be ig- nored; but a new file which has been added with svn add will get added to the repository.

  This is no different from what happens if you use commit -a: new, unknown files are ignored, but new files which have been added with add are added to the repository, as are any changes to existing files. This interaction is nearly identical with the Subversion way of doing things.

The real difference is that in the Subversion case, your “list of next actions” is always deter- mined by looking at the current working tree. In Git, the “list of next actions” is the contents of the index, which represents what will become the next state of HEAD, and that you can manipu- late directly before executing commit. This gives you an extra layer of control over what’s going to happen, by allowing you to stage those changes in advance.

5.1 Reset

- **Doing a mixed reset**: If you use the –mixed option (or no option at all, as this is the default), reset will revert parts of your index along with your HEAD reference to match the given commit. The main difference from –soft is that –soft only changes the meaning of HEAD and doesn't touch the index.

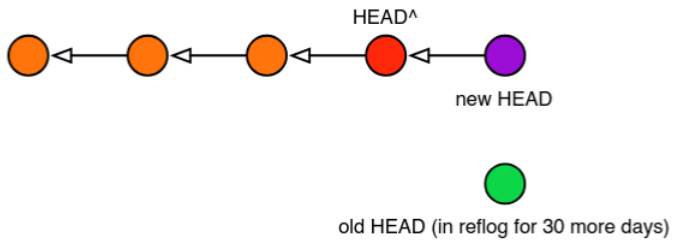
```
git add foo.c # add changes to the index as a new blob
git reset HEAD # delete any changes staged in the index
git add foo.c # made a mistake, add it back
```

- **Doing a soft reset**: If you use the –soft option to reset, this is the same as simply changing your HEAD reference to a different commit. Your working tree changes are left untouched. This means the following two commands are equivalent:

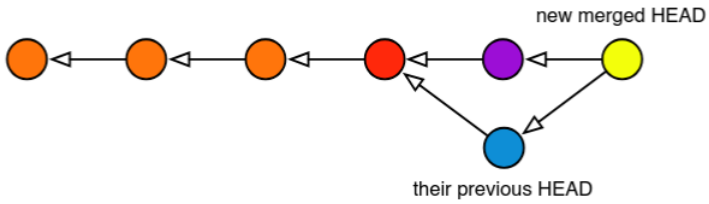
```
git reset --soft HEAD^

git update-ref HEAD HEAD^ # does the same thing, albeit
↪ manually
```

In both cases, your working tree now sits on top of an older HEAD, so you should see more changes if you run status. It's not that your file have been changed, simply that they are now being compared against an older version. It can give you a chance to create a new commit in place of the old one. In fact, if the commit you want to change is the most recent one checked in, you can use commit –amend to add your latest changes to the last commit as if you'd done them together

But please note: if you have downstream consumers, and they've done work on top of your previous head — the one you threw away — changing HEAD like this will force a merge to happen automatically after their next pull. Below is what your tree would look like after a soft reset and a new commit:



And here's what your consumer's HEAD would look like after they pulled again, with colors to show how the various commits match up:



- **Doing a hard reset**: A hard reset (the –hard option) has the potential of being very dangerous, as it's able to do two different things at once: First, if you do a hard reset against your current HEAD, it will erase all changes in your working tree, so that your current files match the contents of HEAD

  There is also another command, `checkout`, which operates just like reset –hard if the in- dex is empty. Otherwise, it forces your working tree to match the index.

  Now, if you do a hard reset against an earlier commit, it's the same as first doing a soft reset and then using reset –hard to reset your working tree. Thus, the following commands are equivalent:

```
git reset --hard HEAD~3 # Go back in time, throwing away
↪ changes

git reset --soft HEAD~3 # Set HEAD to point to an earlier
↪ commit
git reset --hard # Wipe out differences in the working tree
```

  As you can see, doing a hard reset can be very destructive. Fortunately, there is a safer way to achieve the same effect, using the Git stash

```
git stash
git checkout -b new-branch HEAD~3
```

  This approach has two distinct advantages if you're not sure whether you really want to modify the current branch just now

  1. It saves your work in the stash, which you can come back to at any time. Note that the stash is not branch specific, so you could potentially stash the state of your tree while on one branch, and later apply the differences to another.
  2. It reverts your working tree back to a past state, but on a new branch, so if you decide to commit your changes against the past state, you won't have altered your original branch

  If you do make changes to new-branch and then decide you want it to become your new master branch, run the following commands:

```
git branch -D master # goodbye old master (still in reflog)
git branch --move new-branch master # the new-branch is now
↪ my master
```

The moral of this story is: although you can do major surgery on your current branch using reset –soft and reset –hard (which changes the working tree too), why would you want to? Git makes working with branches so easy and cheap, it's almost always worth it to do your destructive modifications on a branch, and then move that branch over to take the place of your old master.

And what if you do accidentally run reset –hard, losing not only your current changes but also removing commits from your master branch? Well, unless you've gotten into the habit of using stash to take snapshots Well, unless you've gotten into the habit of using stash to take snapshots (see next section), there's nothing you can do to recover your lost working tree. But you can restore your branch to its previous state by again using reset – hard with the reflog

```
git reset --hard HEAD@{1}
```

To be on the safe side, never use reset –hard without first running stash. It will save you many white hairs later on. If you did run stash, you can now use it to recover your working tree changes as well

```
git stash
git reset --hard HEAD~3

git reset --hard HEAD@{1}
git stash apply
```

Note: Consider the situation

```
git reset --hard HEAD~3
```

while on `master`. That command does three things:

1. Moves the current branch pointer backward to HEAD 3.
2. Resets the index to match that older commit.
3. Resets the working tree to match that older commit, discarding uncommitted changes.

So suppose your history was:

```
A -- B -- C -- D -- E   master
^
HEAD
```

If you run

```
git reset --hard C
```

Then, master is moved back to C.

```
A -- B -- C   master
^
HEAD

D -- E    no longer referenced by master
```

The commits D and E are not immediately deleted from Git's object database. But they are now unreachable from the branch name master. That is what the text means by "removing commits from your master branch."

Of course, the remote branch may still point to E. In that case,

```
git reset --hard origin/master
```

Recovers. Also, commits that have a tag are reachable. Any reference to a commit keeps that commit reachable. That includes

- branches
- tags
- remote-tracking branches
- HEAD
- stash refs
- reflog entries, temporarily

Stashing and the reflog

- **Intro**: Until now we've described two ways in which blobs find their way into Git: first they're created in your index, both without a parent tree and without an owning commit; and then they're committed into the repository, where they live as leaves hanging off of the tree held by that commit. But there are two other ways a blob can dwell in your repository.

  Note: Recall that the index contains **blob references**, not **tree objects**. Normally the tree object is created when you commit.

  More precisely, each normal index entry is roughly:

  ```
  path              mode        blob object ID
  src/main.c        100644      <hash-of-blob>
  src/util.c        100644      <hash-of-blob>
  README.md         100644      <hash-of-blob>
  ```

  After

  ```
  git add src/main.c
  ```

  Git writes the file contents into the object database as a blob, then records in the index. The index does not need an explicit directory entry like src/. Directories are inferred from paths. When you commit, Git uses the index to construct tree objects, the commit then points to the root of the tree.

- **The reflog**: The first of these is the Git reflog, a kind of meta-repository that records — in the form of commits — every change you make to your repository. This means that when you create a tree from your index and store it under a commit (all of which is done by commit), you are also inadvertently adding that commit to the reflog, which can be viewed using the following command:

  ```
  git reflog

  5f1bc85... HEAD@{0}: commit (initial): Initial commit
  ```

  The beauty of the reflog is that it persists independently of other changes in your repository. This means I could unlink the above commit from my repository (using reset), yet it would still be referenced by the reflog for another 30 days, protecting it from garbage collection. This gives me a month's chance to recover the commit should I discover I really need it.

- **Stash**: The other place blobs can exist, albeit indirectly, is in your working tree itself. What I mean is, say you've changed a file foo.c but you haven't added those changes to the index yet. Git may not have created a blob for you, but those changes do exist, meaning the content exists — it just lives in your filesystem instead of Git's repository. The file even has its own SHA1 hash id, de- spite the fact no real blob exists. You can view it with this command:

```
git hash-object foo.c
<some hash id>
```

What does this do for you? Well, if you find yourself hacking away on your working tree and you reach the end of a long day, a good habit to get into is to stash away your changes:

```
git stash
```

This takes all your directory's contents — including both your working tree, and the state of the index — and creates blobs for them in the git repository, a tree to hold those blobs, and a pair of stash commits to hold the working tree and index and record the time when you did the stash.

This is a good practice because, although the next day you'll just pull your changes back out of the stash with stash apply, you'll have a reflog of all your stashed changes at the end of every day. Here's what you'd do after coming back to work the next morning (WIP here stands for "Work in progress"):

```
git stash list
stash@{0}: WIP on master: 5f1bc85... Initial commit

git reflog show stash # same output, plus the stash commit's
↪ hash id
2add13e... stash@{0}: WIP on master: 5f1bc85... Initial
↪ commit

git stash apply
```

Because your stashed working tree is stored under a commit, you can work with it like any other branch — at any time! This means you can view the log, see when you stashed it, and checkout any of your past working trees from the moment when you stashed them:

```
git stash list
stash@{0}: WIP on master: 73ab4c1... Initial commit
...
stash@{32}: WIP on master: 5f1bc85... Initial commit

git log stash@{32} # when did I do it?
git show stash@{32} # show me what I was working on

git checkout -b temp stash@{32} # let's see that old working
↪ tree!
```

This last command is particularly powerful: behold, I'm now playing around in an uncom- mitted working tree from over a month ago. I never even added those files to the index; I just used the simple expedient of calling stash before logging out each day (provided you actually had changes in your working tree to stash), and used stash apply when I logged back in.

If you ever want to clean up your stash list — say to keep only the last 30 days of activity — don't use stash clear; use the reflog expire command instead:

```
git stash clear # BAD, loose all history

git reflog expire --expire=30.days refs/stash
<outputs the stash bundles that've been kept>
```

Further insight

7.1 Git rm

- **Remove tracked files from remote**: Suppose we have a directory

```
somedir/
```

Which is currently on the remote branch. If we add this directory to .gitignore and push, we will still see it on the remote. We must remove it from the index, then push.

```
git rm -r --cached somedir/
git push
```

Using –cached lets us remove the directory from the index while keeping our working tree the same. -r is the recursive flag.